

GitHub Actions Multi-Environment Lab

— Future Reference

A concise reference for the GitHub Actions CI/CD lab using reusable workflows, DEV/QA/PROD environments, environment variables, secrets, and production approvals.

1. Repository structure

```
.github/  
└─ workflows/  
    ├── main.yml  
    ├── build-test.yml  
    └─ deploy.yml
```

Important: reusable workflows must be directly under `.github/workflows/`; do not place them inside a `templates/` subdirectory.

2. main.yml = pipeline orchestration

Its job is to control the flow: Build/Test → DEV → QA → PROD.

Example:

```
jobs:  
  build-test:  
    uses: ../github/workflows/build-test.yml  
  
  deploy-dev:  
    needs: build-test  
    uses: ../github/workflows/deploy.yml  
    with:  
      environment: dev  
  
  deploy-qa:  
    needs: deploy-dev  
    uses: ../github/workflows/deploy.yml  
    with:  
      environment: qa  
  
  deploy-prod:  
    needs: deploy-qa  
    uses: ../github/workflows/deploy.yml  
    with:  
      environment: prod
```

3. needs controls sequencing

needs: build-test means the dependent job runs only after build-test succeeds.

The resulting flow is: build-test → deploy-dev → deploy-qa → deploy-prod.

4. Reusable workflow = template concept

build-test.yml and deploy.yml use on: workflow_call:.

Call them with uses: ./github/workflows/deploy.yml.

They are reusable workflows called by main.yml rather than ordinary push-triggered workflows.

5. Pass values using with

main.yml passes an input such as with: environment: dev.

The reusable workflow declares workflow_call inputs and accesses the value through `${{ inputs.environment }}`.

Mental model: main.yml → environment input → reusable workflow → selected GitHub Environment.

6. GitHub Environments

The lab uses three environments: dev, qa, prod.

Each environment can have its own variables, secrets, deployment protection rules, required reviewers, and environment-specific configuration.

Example: dev → APP_ENV=dev; qa → APP_ENV=qa; prod → APP_ENV=prod.

7. Environment variables

Use `${{ vars.APP_ENV }}` for environment configuration.

The value changes according to the selected GitHub Environment.

8. Secrets

Use `${{ secrets.SECRET_NAME }}` for sensitive information.

Typical examples: API keys, passwords, cloud credentials, SSH keys, and database credentials.

Never hard-code secrets in YAML or application code.

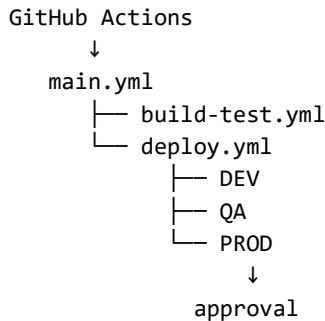
9. Production approval

The prod environment has a required reviewer.

The production flow is: Build → DEV → QA → PROD approval → PROD.

The deployment pauses before PROD until the required reviewer approves it.

10. Big-picture architecture



Reusable deployment logic is shared while environment-specific configuration comes from the selected GitHub Environment.

11. Azure DevOps → GitHub Actions mapping

Azure DevOps azure-pipelines.yml → GitHub Actions main.yml

Azure DevOps template → GitHub reusable workflow

Azure DevOps stage → GitHub job/environment flow

Azure DevOps variables → GitHub vars / env

Azure DevOps variable groups → GitHub variables/secrets

Azure DevOps secret variables → GitHub secrets

Azure DevOps environment → GitHub Environment

Azure DevOps approvals → GitHub Environment required reviewers

Azure DevOps dependsOn → GitHub needs

Azure DevOps pipeline task → GitHub Action / run step

Azure DevOps agent pool → GitHub runs-on

12. Eight key lessons

1. main.yml = orchestrator
2. workflow_call = reusable workflow
3. uses = call reusable workflow
4. needs = dependency/order
5. with = pass inputs
6. vars = configuration
7. secrets = sensitive configuration
8. environment protection = production approval

Final mental model

Do not duplicate the pipeline for DEV, QA, and PROD.

Create reusable workflows once, pass the environment as an input, and let GitHub Environment configuration provide environment-specific values.

This pattern can be reused for AWS, Azure, or GCP deployment pipelines.

Action items / next steps

1. Keep main.yml as the orchestration layer.
2. Keep build-test.yml reusable and focused on CI.
3. Keep deploy.yml reusable and parameterized by environment.
4. Maintain separate dev, qa, and prod GitHub Environments.
5. Put non-sensitive configuration in environment variables.
6. Put credentials and sensitive values in environment secrets.
7. Protect PROD with required reviewers.
8. Before real cloud deployment, test that DEV and QA use their own configuration and PROD pauses for approval.
9. Later replace the demo deployment step with the chosen AWS, Azure, or GCP deployment action.